



1. Handle Missing Values

[40]:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({'Age': [25, None, 30]})
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
df['Age'] = df['Age'].fillna(df['Age'].mean())
print(df)
```

	Age
0	25.0
1	27.5
2	30.0

@ clcoding

2. Drop Unnecessary Columns

[9]:

```
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4], 'C': [5, 6]})
df.drop(columns=['C'], inplace=True)
print(df)
```

	A	B
0	1	3
1	2	4



3. Encode Categorical Data

[10]:

```
df = pd.DataFrame({'color': ['red', 'blue', 'red']})
df['color_code'] = df['color'].map({'red': 0, 'blue': 1})
print(df)
```

	color	color_code
0	red	0
1	blue	1
2	red	0

4. Normalize / Scale Features

[11]:

@ clcoding

```
from sklearn.preprocessing import MinMaxScaler

data = [[10], [20], [30]]
scaler = MinMaxScaler()
print(scaler.fit_transform(data))
```

```
[[0. ]
 [0.5]
 [1. ]]
```

/clcoding



/Pythoncoding



/Pythonclcoding





5. Standardize Features

[12]:

```
from sklearn.preprocessing import StandardScaler

data = [[1],[2],[3]]
scaler = StandardScaler()
print(scaler.fit_transform(data))
```

```
[[-1.22474487]
 [ 0.          ]
 [ 1.22474487]]
```

6. Remove Duplicates

[13]:

@ clcoding

```
df = pd.DataFrame({'A':[1,2,2,3]})
df = df.drop_duplicates()
print(df)
```

```
   A
0  1
1  2
3  3
```



7. Split Data into Train/Test

[15]:

```
from sklearn.model_selection import train_test_split

X = [[1],[2],[3],[4]]
y = [0,0,1,1]
X_train, X_test, y_train,
y_test = train_test_split(X,y,test_size=0.5,
                          random_state=42)
print(X_train, X_test)
```

```
[[1], [3]] [[2], [4]]
```

@ clcoding

8. Create Dummy Variables

[16]:

```
df = pd.DataFrame({'color':['red','blue']})
df_dummies = pd.get_dummies(df, columns=['color'])
print(df_dummies)
```

	color_blue	color_red
0	False	True
1	True	False

/clcoding



/Pythoncoding



/Pythonclcoding





9. Detect Outliers (Simple Z-Score)

[18]:

```
import numpy as np

data = np.array([10,12,12,13,100])
z = (data - data.mean())/data.std()
print(np.where(np.abs(z)>2))
```

(array([], dtype=int64),)

10. Text Preprocessing (Lowercase + Strip)

@ clcoding

[19]:

```
texts = [" Hello ", "WORLD "]
cleaned = [t.strip().lower() for t in texts]
print(cleaned)
```

['hello', 'world']

/clcoding



/Pythoncoding



/Pythonclcoding

